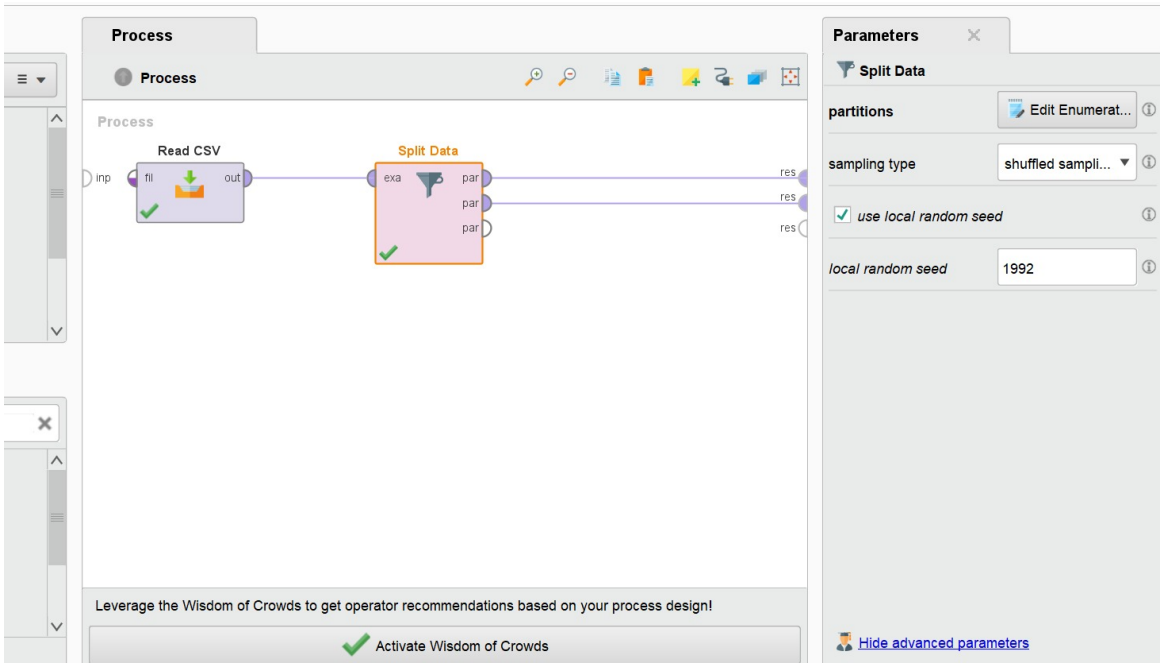
Predicting Babar Azam's Runs in Altair AI Studio

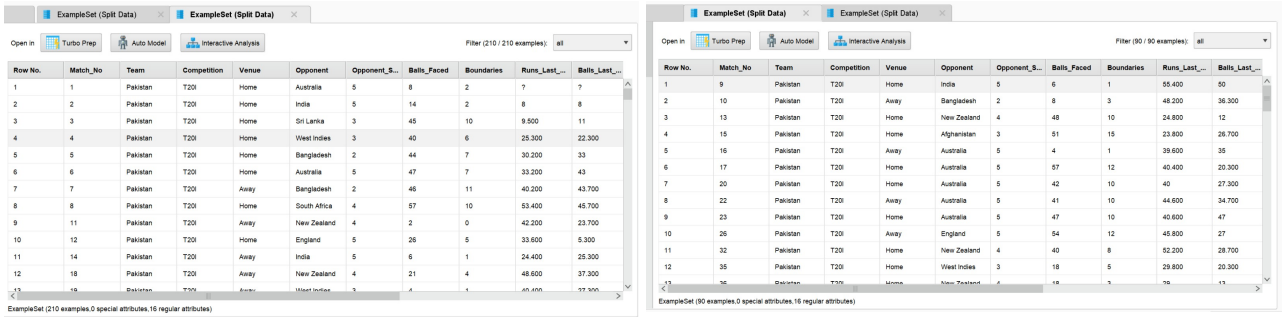
Goal: predict the **Runs_Category** column (Low, Medium, High) from the other (known) columns. Steps: split the data, build a Decision Tree, then a Random Forest.

Step 1: Split the Data

The data was imported with **Read CSV** and connected to **Split Data**. Its parameters use **shuffled sampling** (random rows) and two partitions set with Edit Enumeration, which divide the 300 matches into **70% training data (210 matches)** and **30% testing data (90 matches)**. The local random seed (1992) keeps the same split every time it runs.



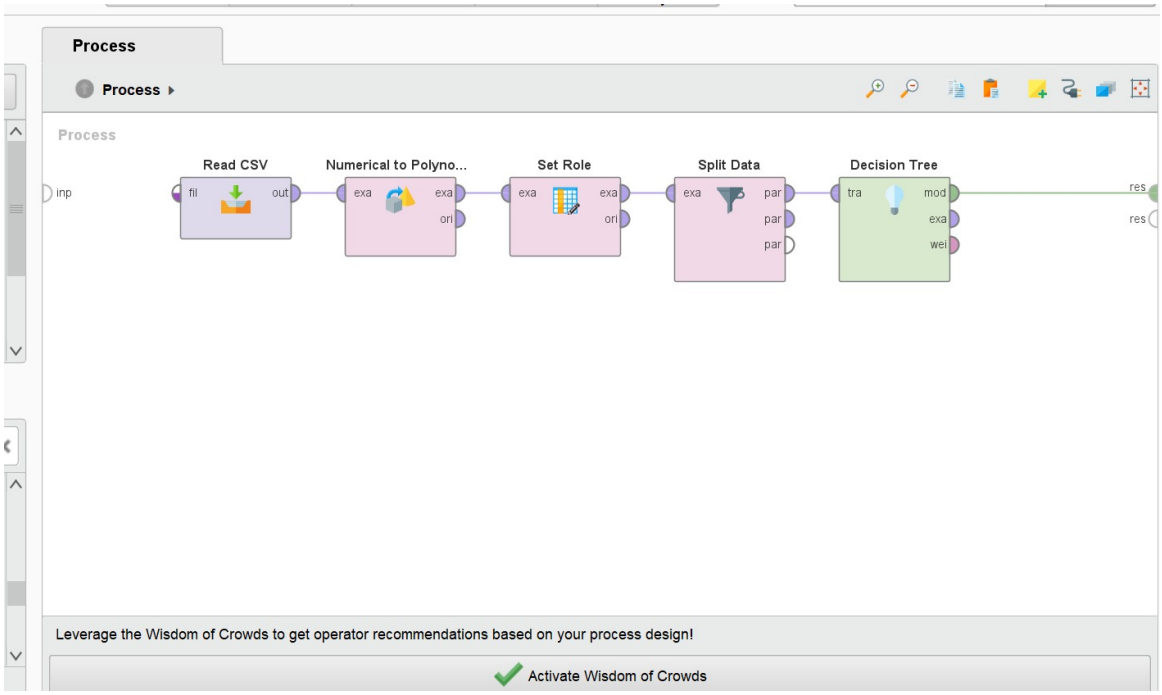
Split Data design and its parameters.



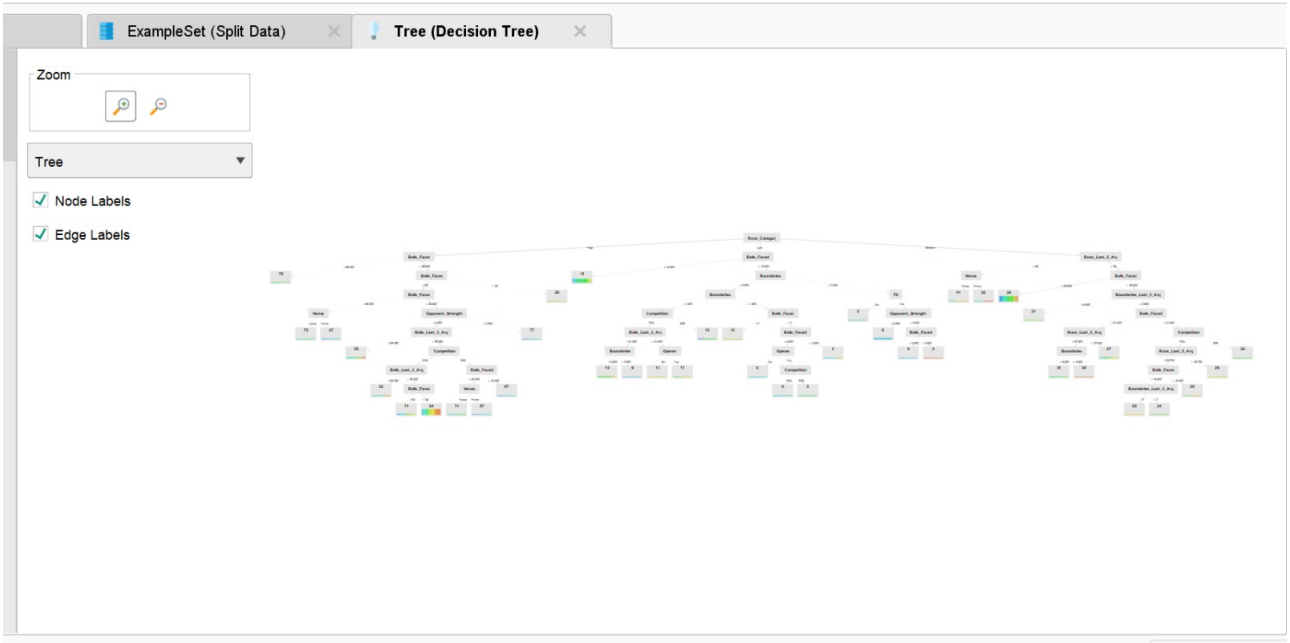
Left: training data (210 examples). Right: testing data (90 examples).

Step 2: Decision Tree

Numerical to Polynominal turns the run categories into classes (Low, Medium, High) and **Set Role** marks Runs_Category as the label (what we predict). The training part (70%) from Split Data is then connected to the **Decision Tree** operator, and its model output is sent to the results. The tree learns rules from columns such as Balls_Faced, Boundaries, Opponent_Strength and Runs_Last_3_Avg to decide how many runs Babar Azam is likely to score.



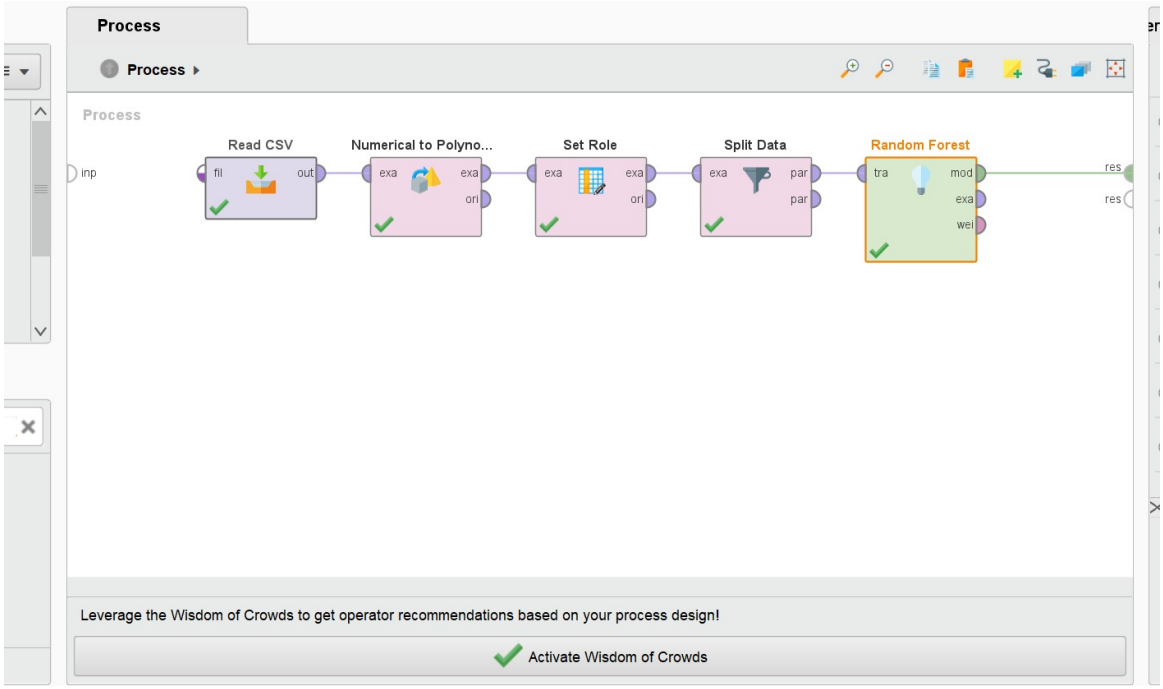
Process design: Read CSV, Numerical to Polynominal, Set Role, Split Data, Decision Tree.



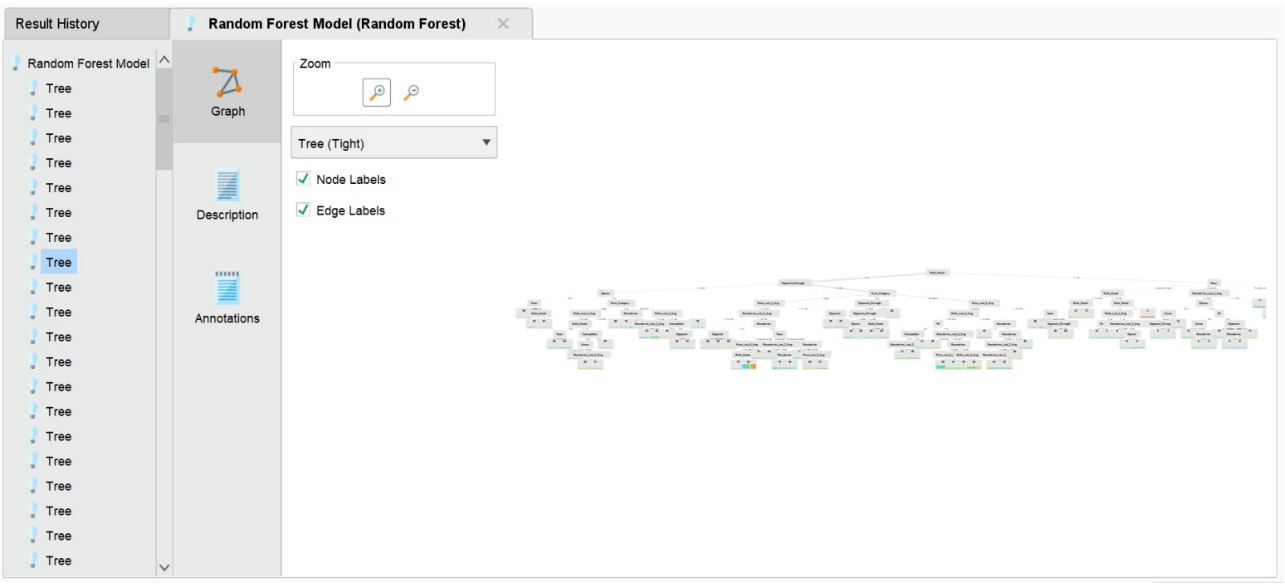
Resulting decision tree (Graph view). Each box is a question about a column and each end box is a predicted Runs_Category value.

Step 3: Random Forest

Random Forest replaces the Decision Tree in the same process. Instead of one tree it builds many trees, each from a random part of the training data, and combines their results. This is usually more reliable than a single tree.



Process design with Random Forest connected to the training part of Split Data.



Random Forest model: the list on the left shows the individual trees, and the graph shows one selected tree.

Summary: Babar Azam's data was split 70/30 (210 train, 90 test), then a Decision Tree (one tree) and a Random Forest (many trees) were trained to predict Runs_Category.